



NanoPulldown: a Local *In Silico* Pulldown Screen

AlphaPulldown-style bait/prey screening with ESMFold2-Fast on a single Apple
Silicon Mac Studio

Etienne Reboul

Groupe Gibbérellines et adaptation à l'environnement
Institut de Biologie Moléculaire des Plantes (IBMP), Strasbourg

October 6, 2026

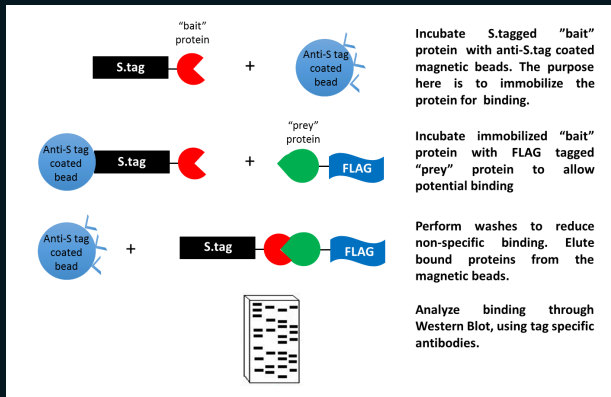
Outline

1. Introduction
2. The pipeline
3. Config and reuse
4. Report
5. Caveats and next steps

Introduction

IP-MS: finding what binds a bait

Background: IP-MS



Pull-down with tagged proteins (S-tag / FLAG). RaihaT, Wikimedia Commons, CC

BY-SA 3.0

1. **Immunoprecipitation:** tagged *bait* captured on beads
2. Incubate with a lysate: *prey* proteins bind
3. Wash, then elute
4. **Mass spectrometry** identifies the eluate

Limits: direct and indirect (complex-mediated) binders are mixed; background binders to the beads / tag

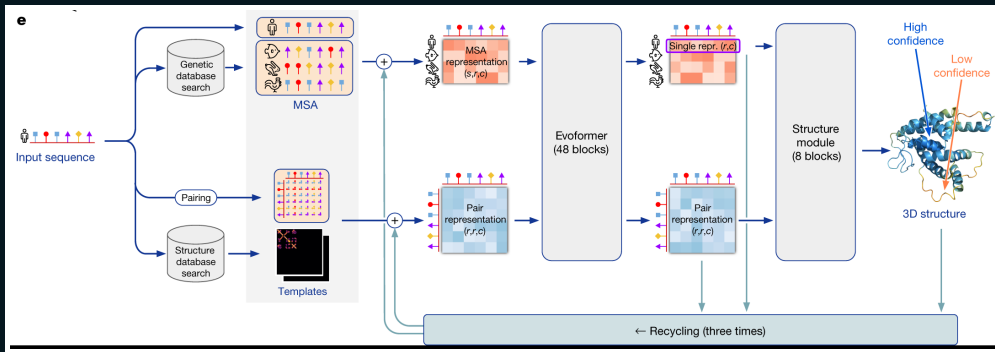
AlphaFold-Multimer

AlphaPulldown

- AlphaFold2 retrained on **complexes**: several chains folded jointly
- Scores the interface with **ipTM** (and pTM / PAE) on top of per-residue pLDDT
- **AlphaPulldown**: bait $\times$ prey list from UniProt IDs $\rightarrow$ one multimer prediction per pair $\rightarrow$ ranked interface scores – an *in silico* pulldown
- Cost: one multimer inference per pair, plus MSA building (below) $\Rightarrow$ GPU cluster

The AlphaFold2 pipeline – it all starts with an MSA

AlphaFold2



Jumper *et al.* 2021, *Nature* (Fig. 1e, cropped); Wikimedia Commons, CC BY 4.0

Genetic database search → MSA (+ pairing for complexes, + templates) is the first and the slowest step: the Evoformer reads co-evolution from it.

ESMFold vs. AF2-Multimer: two ways to use evolution

Language-model

folding

	AF2-Multimer	ESMFold-type (protein LM)
Input	sequence + MSA + templates	single sequence
Evolutionary signal	read explicitly from the alignment at inference	learned in the weights of a language model during pre-training
Pre-processing	database searches (slow, heavy)	none
Structure head	Evoformer + structure module	folding trunk on LM embeddings → structure module / diffusion
Weak spot	few homologs ⇒ poor MSA	orphan / rapidly evolving sequences: relies on what the LM memorized

Zoom on ESMFold2-Fast

Language-model folding

Candido, Hayes, Derry, Rao, Lin, Verkuil *et al.*, Biohub / EvolutionaryScale – *Language Modeling Materializes a World Model of Protein Biology* (bioRxiv, 2026)

- **ESMFold2**: recurrent folding trunk on **frozen ESMC** representations (pooled over all layers), diffusion transformer for atomic coordinates
- **Fast variant**: 24 folding layers instead of 48, built for high-throughput screening and design
- **No MSA**: sequence-only prediction competitive with MSA-based methods
- Reported: protein–protein $68 \pm 1 \%$ pass rate; antibody–antigen $50 \pm 2 \%$ (AF3: 47 %)
- ~ 9.4 s for 1 024 residues on one H100 (reported $1.3\times$ faster than AF3)

⇒ **This is what makes a laptop pulldown screen feasible**

Motivation

Motivation

- A pulldown / IP-MS experiment returns a **long list** of candidate interactors – which of them bind the bait *directly*?
 - Worked case: **BPM6** (AT3G43700, CUL3 substrate adaptor), GFP-IP in *A. thaliana* (PXD013906, Chico *et al.* 2020) – 1 bait $\times$ $\sim$ 1 500 prey candidates
 - AlphaPulldown's UniProt-ID-driven bait/prey strategy answers this, but assumes AlphaFold-multimer/AF3, MSAs and a GPU cluster
- ⇒ Can we run the same screen **on one laptop**?

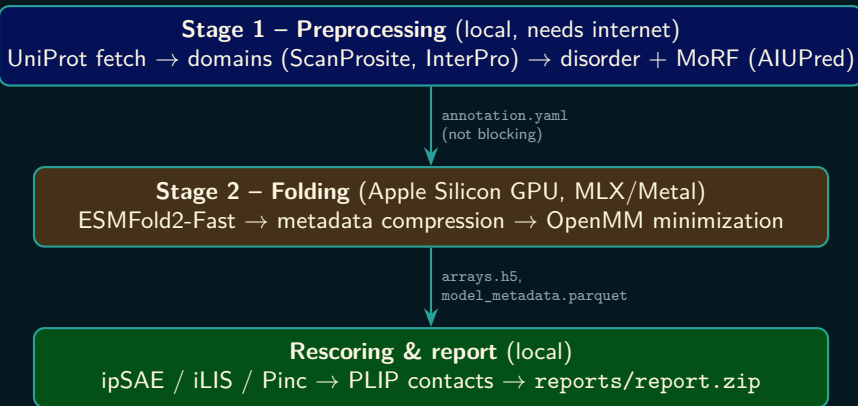
Design choices

Motivation

1. **ESMFold2-Fast** (MLX/Metal): no MSA, no templates, no cluster
2. **Protein–protein only**: one seed, 1 diffusion sample per pair, no pose clustering
3. **One command, one report**: a single Snakemake rule all from sequence annotation to the final report
4. Adding pairs = a bait list + a prey list; pair specs are **generated**, not hand-written

The pipeline

Two stages, one diagram



Everything runs on one machine, in one Snakemake call.

Stage 1 – Preprocessing

- **Pair expansion:** `configs/baits.txt` $\times$ `configs/preys.txt` (AlphaPulldown syntax: `ID`, `ID:N`, `ID:start-stop`) $\rightarrow$ `configs/<bait>__<prey>.yaml`; pairs over `esmfold2.max_tokens` are skipped and listed
 - **Domain annotation:** ScanProsite + InterPro API (fast path by accession; InterProScan v6 MATCH as fallback)
 - **Disorder + MoRF:** AIUPred (docker), one container run per 200 sequences
 - Annotation runs **once per unique sequence** (cached) with polite, ramped rate limits on the public APIs
- $\Rightarrow$ proposed domains: block, curated **whenever convenient** – it never blocks folding

Stage 2 – Folding

- **ESMFold2-Fast** (biohub/ESMFold2-Fast), MLX on Metal: 10 loops, 100 sampling steps, 1 seed, 1 diffusion sample
- Memory-bound, not compute-bound: ~ 25 GB model load in unified memory $\Rightarrow$ `--resources gpu=1` is **mandatory**
- **Batched folding**: 50 pairs per model load, so the model is loaded once per chunk, not once per pair
- **Token limit**: peak memory grows $\sim n^{2.2}$; measured on a 36 GB M4 Max, 1 000 tokens ≈ 29 GB, 145 s (`max_tokens: 1000`)
- Metadata compression: pair ipTM / PAE $\rightarrow$ `arrays.h5` + `model_metadata.parquet`

Minimization, rescoring and contacts

1. **OpenMM minimization** (`amber14` + GBn2 implicit solvent; OpenCL with CPU fallback; divergence detection and failure table)
2. **Rescoring**: ipSAE, iLIS, Pinc (PAE cutoff 12 Å, $C\beta$ contact cutoff 8 Å) next to ESMFold2's own pair ipTM
3. **PLIP** (`docker, linux/amd64` under Rosetta): bait $\leftrightarrow$ prey contacts per minimized sample, merged into `plip_summary.csv` per pair
4. PAE-based **domain** $\times$ **domain confidence heatmap** per sample

Running it

```
conda env create -f envs/controller.yaml
python scripts/expand_pairs.py      # baits x preys -> configs/<pair>.yaml
# add the new pair stems to config.yaml's pairs: list
snakemake --use-conda --cores 4 --resources gpu=1 docker_heavy=3
```

- `gpu=1`: caps `run_esmfold2` at one model load – nothing else guards against concurrent loads (OOM)
- `docker_heavy=N`: keeps minimization / PLIP containers from running during a model load, then lets N run in parallel

⇒ **Output:** `one reports/report.zip`

Config and reuse

Adding a new screen

Edit **only**: the bait/prey lists, then one line per pair in `config.yaml`'s pairs:. **No script edits.**

file	role
<code>config.yaml</code>	shared defaults, both stages
<code>config.local.yaml</code>	per-machine overrides (gitignored): InterPro e-mail, <code>max_tokens</code>
<code>configs/baits.txt</code> , <code>preys.txt</code>	one protein spec per line
<code>configs/<bait>__<prey>.yaml</code>	chain spec + curated domains (generated)
<code>envs/</code>	one conda env per rule group (<code>--use-conda</code>)

Reused from `ab_initio_pipeline`

- Vended **ipSAE** and **Pinc** (`tools/`), unchanged
- Same dark `custom.css` report theme and `datavzrd` tables
- Same annotation logic (domains, disorder), adapted: **AIUPred** replaces **IUPred3/ANCHOR2** **and** **MoRFchibi2** in one step
- **Replaced**: no HPC scheduler, no ABCfold backends, no MSA, no ChimeraX / antechamber (OpenMM instead), no pose clustering

Report

One report, zipped

- Snakemake report, **SVG/PDF figures only**, dark theme (report/custom.css)
- **Zip, not bare .html**: browsers block the data: URLs needed to embed interactive datavzrd tables → unzip, open report.html
- Contents:
 - results table across all pairs (ipSAE / iLIS / Pinc / pair ipTM)
 - PLIP contact heatmaps, domain-aware
 - PAE domain × domain confidence heatmaps
 - minimization energy plots + failure table
- Stage-1 figures (domain map, disorder / binding) are kept as plain SVGs under data/annotation/<pair>/

Caveats and next steps

Assumptions still to confirm – be honest

- **MLX path:** assumed to fill `result.pae / pair_chains_iptm` like the Torch/CUDA path – to check against a live run
- **AIUPred CLI** (`-i/-t/-o`) and the **InterProScan v6 MATCH** endpoint: to confirm against current docs
- **OpenMM OpenCL** on Metal: CPU fallback exists, but initialization is untested on other Macs
- **PLIP on Apple Silicon:** amd64 emulation is slow and needs Rosetta in Docker Desktop
- **Memory:** 3 concurrent PLIP containers once caused a swap/OOM crash $\Rightarrow$ start with `docker_heavy=2`

Roadmap

- Complete the BPM6 screen over the PXD013906 prey list and compare ranking against the PNAS Dataset S1 known / significant interactors
- Benchmark ESMFold2-Fast ranking metrics (ipSAE, iLIS, pair ipTM) on known positives vs. random preys
- Tune `max_tokens` and `pairs_per_load` per machine (`notebooks/esmfold2_memory_model.ipynb`)
- Optional: re-fold top hits with AF3 / ABCfold for confirmation

Questions?